

Telco Customer Churn Dataset Overview

The **Telco Customer Churn** dataset contains information about customers of a fictional telecom company. The objective is to predict whether a customer will **churn (leave the service)** based on their demographic, service usage, and account details.

Source:

Originally available as part of IBM sample datasets, the **Telco Customer Churn** dataset is now publicly hosted on Kaggle and GitHub.

[-Kaggle Dataset Page](#)

[-GitHub CSV File](#)

Target Variable:

- **Churn** (*binary*): Whether the customer has churned (Yes or No)

Features:

Column Name	Description	Data Type
customerID	Unique customer identifier	String
gender	Gender (Male , Female)	Categorical
SeniorCitizen	Whether the customer is a senior citizen (0 , 1)	Categorical
Partner	Has a partner (Yes , No)	Categorical
Dependents	Has dependents (Yes , No)	Categorical
tenure	Number of months the customer has stayed	Numeric (int)
PhoneService	Has phone service (Yes , No)	Categorical
MultipleLines	Has multiple lines (Yes , No , No phone service)	Categorical
InternetService	Type of internet service (DSL , Fiber optic , No)	Categorical
OnlineSecurity	Has online security (Yes , No , No internet service)	Categorical
OnlineBackup	Has online backup (Yes , No , No internet service)	Categorical
DeviceProtection	Has device protection (Yes , No , No internet service)	Categorical
TechSupport	Has technical support (Yes , No , No internet service)	Categorical
StreamingTV	Streams TV (Yes , No , No internet service)	Categorical
StreamingMovies	Streams movies (Yes , No , No internet service)	Categorical
Contract	Type of contract (Month-to-month , One year , Two year)	Categorical
PaperlessBilling	Uses paperless billing (Yes , No)	Categorical

Column Name	Description	Data Type
PaymentMethod	Payment method (Electronic check , Mailed Check , bank Transfer , Credit Card)	<i>Categorical</i>
MonthlyCharges	The amount charged per month	<i>Numeric (float),</i>
TotalCharges	Total amount charged	<i>Numeric (float, needs cleaning)</i>
Churn	Whether the customer left	Categorical (target)

Python Packages Used

1. pandas

- **Purpose:** Loading, cleaning, and manipulating the dataset
- **Import:** `import pandas as pd`
- **Install:** `pip install pandas`

2. matplotlib

- **Purpose:** Visualizing, plotting graphs and decision trees
- **Import:** `import matplotlib.pyplot as plt`
- **Install:** `pip install matplotlib`

3. scikit-learn (sklearn)

- **Purpose:** Data preprocessing, model building, evaluation
- **Install:** `pip install scikit-learn`

Modules used:

- `sklearn.preprocessing`
 - `KBinsDiscretizer` – discretizing numeric features
- `sklearn.model_selection`
 - `train_test_split` – splitting dataset into train/test sets
- `sklearn.tree`
 - `DecisionTreeClassifier` – training decision trees
 - `plot_tree` – visualizing trained decision trees
- `sklearn.ensemble`
 - `RandomForestClassifier` – training random forest models
- `sklearn.metrics`
 - `accuracy_score` – evaluating accuracy
 - `precision_score` – evaluating precision
 - `recall_score` – evaluating recall
 - `f1_score` – evaluating F1 score
 - `roc_auc_score` – computing AUC for classification

- classification_report – detailed per-class evaluation

4. numpy

- **Purpose:** Numerical operations in plotting and evaluation
- **Import:** import numpy as np
- **Install:** pip install numpy

```
In [ ]: # Install packages at once
        # !pip install pandas matplotlib scikit-learn numpy
```

Loading the dataset into a dataframe

```
In [ ]: import pandas as pd

        # Load the Telco Customer Churn dataset
        df = pd.read_csv('WA_Fn-UseC_-Telco-Customer-Churnvx1.csv')

        df
```

```
Out[ ]:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service
1	5575-GNVDE	Male	0	No	No	34	Yes	No
2	3668-QPYBK	Male	0	No	No	2	Yes	No
3	7795-CFOCW	Male	0	No	No	45	No	No phone service
4	9237-HQITU	Female	0	No	No	2	Yes	No
...	...	...	...	...	...	...	...	...
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Yes
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	Yes
7040	4801-JJAZL	Female	0	Yes	Yes	11	No	No phone service
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes
7042	3186-AJIEK	Male	0	No	No	66	Yes	No

7043 rows × 21 columns



Column names with their data types

In []:

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7043 non-null   object
1   gender                 7043 non-null   object
2   SeniorCitizen          7043 non-null   int64
3   Partner                7043 non-null   object
4   Dependents             7043 non-null   object
5   tenure                 7043 non-null   int64
6   PhoneService           7043 non-null   object
7   MultipleLines          7043 non-null   object
8   InternetService        7043 non-null   object
9   OnlineSecurity         7043 non-null   object
10  OnlineBackup           7043 non-null   object
11  DeviceProtection       7043 non-null   object
12  TechSupport            7043 non-null   object
13  StreamingTV            7043 non-null   object
14  StreamingMovies        7043 non-null   object
15  Contract               7043 non-null   object
16  PaperlessBilling       7043 non-null   object
17  PaymentMethod          7043 non-null   object
18  MonthlyCharges         7043 non-null   float64
19  TotalCharges           7043 non-null   object
20  Churn                  7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

Observation 1

- The column `TotalCharges` appears as `object` in `df.info()` instead of numeric. This is due to the presence of rows with blank string entries (`" "`), which pandas does not treat as missing by default.

In []:

```
print(f'No of samples where TotalCharges is Blank:{df[df["TotalCharges"].str.
# Show rows where TotalCharges is blank
df[df["TotalCharges"].str.strip() == ""][["customerID", "tenure", "TotalCharg
```

No of samples where TotalCharges is Blank:11

Out[]:

	customerID	tenure	TotalCharges
488	4472-LVYGI	0	
753	3115-CZMZD	0	
936	5709-LVOEQ	0	
1082	4367-NUYAO	0	
1340	1371-DWPAZ	0	
3331	7644-OMVMY	0	
3826	3213-VVOLG	0	
4380	2520-SGTTA	0	
5218	2923-ARZLG	0	
6670	4075-WKNIU	0	
6754	2775-SEFEE	0	

```
In [ ]: # convert the column to `float64` and removes invalid rows.
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
df.dropna(subset=["TotalCharges"], inplace=True)
df.shape
```

```
Out[ ]: (7032, 21)
```

Observation 2

- SeniorCitizen is stored as int64, it is a binary categorical feature (0 = No, 1 = Yes). To reflect its actual meaning, we convert it to categorical type

```
In [ ]: df["SeniorCitizen"] = df["SeniorCitizen"].astype("object")
```

```
In [ ]: categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()

print("Categorical Columns:")
print(categorical_cols, len(categorical_cols))
print("\nNumerical Columns:")
print(numerical_cols, len(numerical_cols))
```

Categorical Columns:

['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod', 'Churn'] 18

Numerical Columns:

['tenure', 'MonthlyCharges', 'TotalCharges'] 3

Merging "No internet/phone service" with "No"

- In this dataset, several categorical columns contain values like "No internet service" or "No phone service" in addition to a regular "No".
- These extra values do not carry distinct meaning from "No" in terms of behavior — they simply indicate that the person does not have the service, which already implies "No" for all related options.

```
In [ ]: cols_to_check = [
    "MultipleLines", "OnlineSecurity", "OnlineBackup",
    "DeviceProtection", "TechSupport", "StreamingTV", "StreamingMovies"
]

for col in cols_to_check:
    print(f"\nValue counts for '{col}':")
    print(df[col].value_counts())
```

Value counts for 'MultipleLines':

MultipleLines	
No	3385
Yes	2967

```
No phone service      680
Name: count, dtype: int64
```

```
Value counts for 'OnlineSecurity':
OnlineSecurity
No              3497
Yes            2015
No internet service  1520
Name: count, dtype: int64
```

```
Value counts for 'OnlineBackup':
OnlineBackup
No              3087
Yes            2425
No internet service  1520
Name: count, dtype: int64
```

```
Value counts for 'DeviceProtection':
DeviceProtection
No              3094
Yes            2418
No internet service  1520
Name: count, dtype: int64
```

```
Value counts for 'TechSupport':
TechSupport
No              3472
Yes            2040
No internet service  1520
Name: count, dtype: int64
```

```
Value counts for 'StreamingTV':
StreamingTV
No              2809
Yes            2703
No internet service  1520
Name: count, dtype: int64
```

```
Value counts for 'StreamingMovies':
StreamingMovies
No              2781
Yes            2731
No internet service  1520
Name: count, dtype: int64
```

To simplify the dataset and avoid redundant categories during encoding, we merged:

- "No internet service" → "No"
- "No phone service" → "No"

```
In [ ]: # Merge "No internet service" / "No phone service" with "No"
internet_cols = [
    "OnlineSecurity", "OnlineBackup", "DeviceProtection",
    "TechSupport", "StreamingTV", "StreamingMovies"
]
for col in internet_cols:
    df[col] = df[col].replace("No internet service", "No")

df["MultipleLines"] = df["MultipleLines"].replace("No phone service", "No")

# Step 3: Print value counts after merging
print("\nAfter merging:\n")
for col in cols_to_check:
    print(f"\nValue counts for '{col}':")
    print(df[col].value_counts())
```

After merging:

Value counts for 'MultipleLines':

MultipleLines

No 4065

Yes 2967

Name: count, dtype: int64

Value counts for 'OnlineSecurity':

OnlineSecurity

No 5017

Yes 2015

Name: count, dtype: int64

Value counts for 'OnlineBackup':

OnlineBackup

No 4607

Yes 2425

Name: count, dtype: int64

Value counts for 'DeviceProtection':

DeviceProtection

No 4614

Yes 2418

Name: count, dtype: int64

Value counts for 'TechSupport':

TechSupport

No 4992

Yes 2040

Name: count, dtype: int64

Value counts for 'StreamingTV':

StreamingTV

No 4329

Yes 2703

Name: count, dtype: int64

Value counts for 'StreamingMovies':

StreamingMovies

No 4301

Yes 2731

Name: count, dtype: int64

In []:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 7032 entries, 0 to 7042
```

```
Data columns (total 21 columns):
```

#	Column	Non-Null Count	Dtype
0	customerID	7032 non-null	object
1	gender	7032 non-null	object
2	SeniorCitizen	7032 non-null	object
3	Partner	7032 non-null	object
4	Dependents	7032 non-null	object
5	tenure	7032 non-null	int64
6	PhoneService	7032 non-null	object
7	MultipleLines	7032 non-null	object
8	InternetService	7032 non-null	object
9	OnlineSecurity	7032 non-null	object
10	OnlineBackup	7032 non-null	object
11	DeviceProtection	7032 non-null	object
12	TechSupport	7032 non-null	object
13	StreamingTV	7032 non-null	object
14	StreamingMovies	7032 non-null	object
15	Contract	7032 non-null	object

```

16 PaperlessBilling    7032 non-null    object
17 PaymentMethod      7032 non-null    object
18 MonthlyCharges     7032 non-null    float64
19 TotalCharges        7032 non-null    float64
20 Churn               7032 non-null    object
dtypes: float64(2), int64(1), object(18)
memory usage: 1.2+ MB

```

Heatmap for categorical attributes

- We used Cramér's V to measure the strength of association between categorical features and visualized them using a heatmap.
- Cramér's V is a recommended metric for measuring association between categorical attributes, especially when variables are nominal and do not have a natural order.
- Cramér's V ranges from 0 to 1, where 0 indicates no association and 1 indicates a strong association between categorical variables.

```

In [ ]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import chi2_contingency
import warnings

def cramers_v_basic(x, y):
    confusion_matrix = pd.crosstab(x, y)
    if confusion_matrix.shape[0] < 2 or confusion_matrix.shape[1] < 2:
        return np.nan # not enough variability to compute
    chi2 = chi2_contingency(confusion_matrix)[0]
    n = confusion_matrix.sum().sum()
    return np.sqrt(chi2 / (n * (min(confusion_matrix.shape) - 1 + 1e-10)))

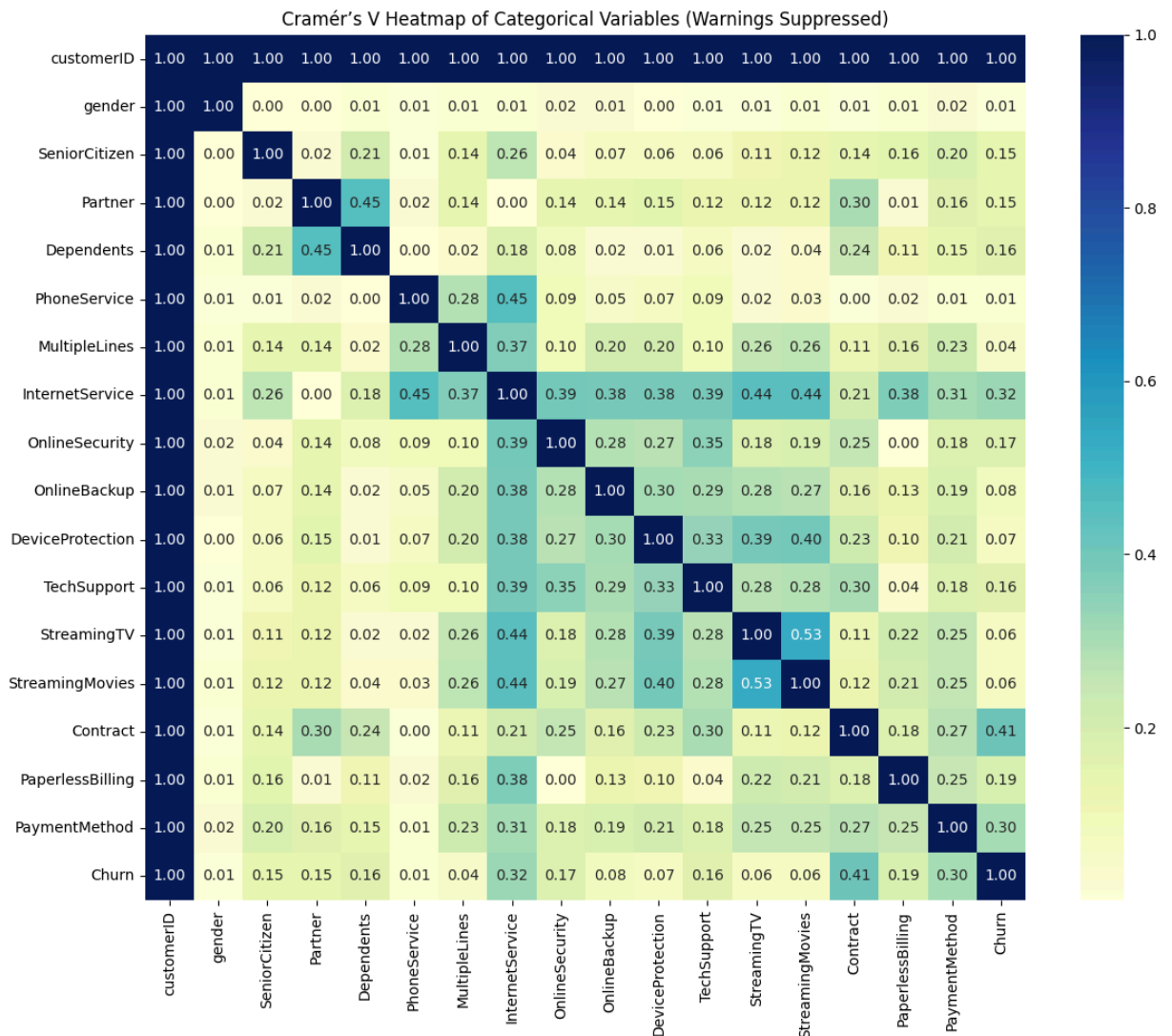
cols = [col for col in categorical_cols if col != 'Churn'] + ['Churn']

cramer_matrix = pd.DataFrame(np.zeros((len(cols), len(cols))), index=cols, co

# Suppress warnings during computation
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    for col1 in cols:
        for col2 in cols:
            if col1 == col2:
                cramer_matrix.loc[col1, col2] = 1.0
            else:
                cramer_matrix.loc[col1, col2] = cramers_v_basic(df[col1], df[

# Plot the heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(cramer_matrix, cmap="YlGnBu", annot=True, fmt=".2f")
plt.title("Cramér's V Heatmap of Categorical Variables (Warnings Suppressed)")
plt.tight_layout()
plt.show()

```

Observations

- **Contract type** has one of the strongest associations with Churn (~0.41), meaning contract duration is an important driver of customer churn.
- **InternetService** and related services (OnlineSecurity, TechSupport, StreamingTV/Movies) show moderate inter-correlations (0.3–0.5), indicating these features are interconnected.
- Most other categorical variables have very low correlations (less than 0.1), suggesting minimal dependency and low redundancy among them.

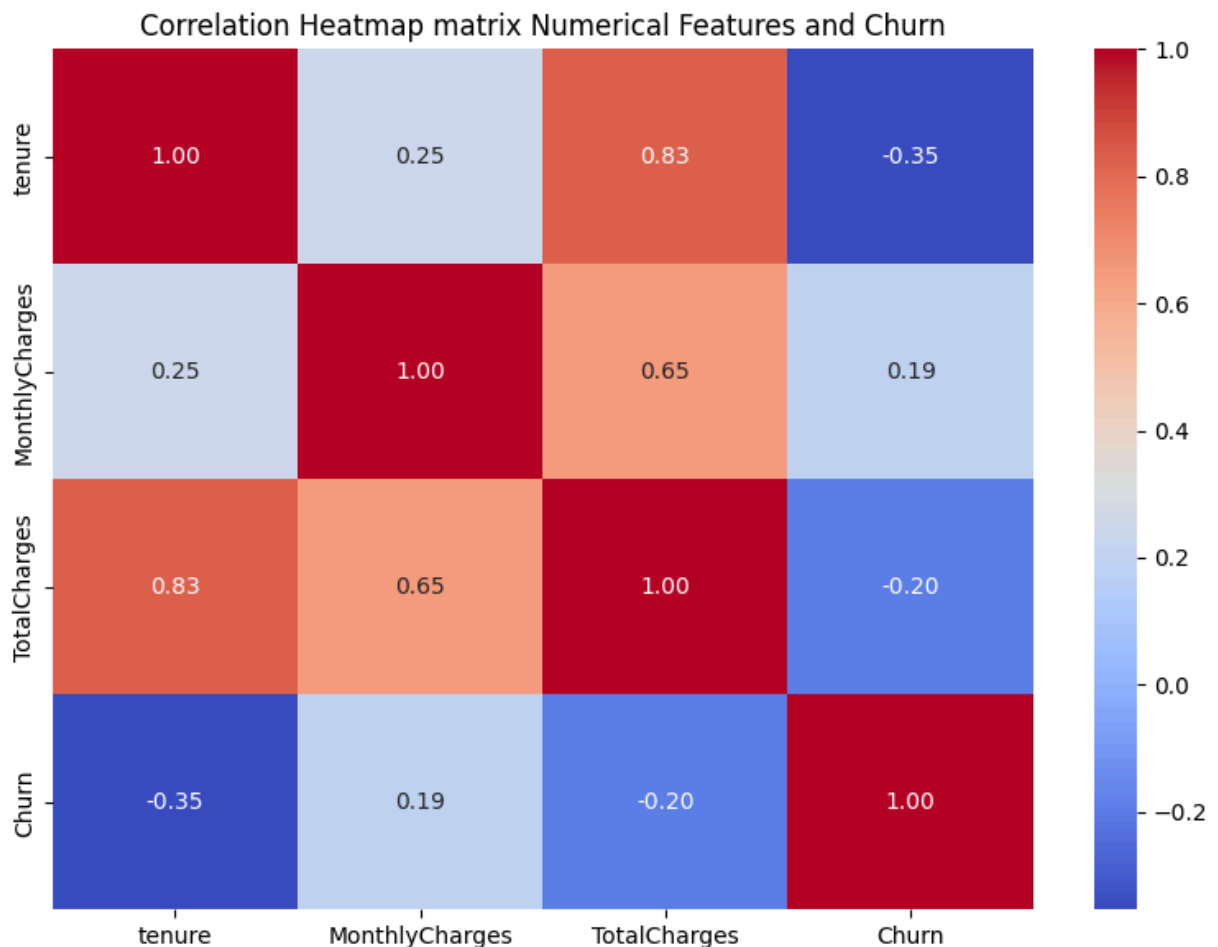
Pearson Correlation Heatmap: Numerical Features vs. Churn

```
In [ ]: import seaborn as sns
import matplotlib.pyplot as plt

df["Churn"] = df["Churn"].map({"Yes": 1, "No": 0}) if df["Churn"].dtype == ob

cols = numerical_cols + ["Churn"]
corr_matrix = df[cols].corr()
```

```
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap matrix Numerical Features and Churn")
plt.tight_layout()
plt.show()
```



- **Tenure** has the strongest relationship with churn (≈ -0.35), indicating that customers with shorter tenure are more likely to churn.

Discretizing numeric attributes

We turned the numeric features `tenure`, `MonthlyCharges`, and `TotalCharges` into 4 categories (bins) each, using a method called **equi-frequency binning**.

- **Equi-frequency binning** means we split the data so that **each bin has about the same number of data points**.
- To do this, we used `KBinsDiscretizer` from `scikit-learn` with the **quantile strategy**, which automatically creates bins with equal numbers of samples.
- We set the number of bins to **4**, so each bin holds roughly **25% of the data**.
- The new columns (`tenure_bin`, `MonthlyCharges_bin`, and `TotalCharges_bin`) contain values from **0 to 3**, showing which bin each value falls into.

```
In [ ]: # resetting churn to yes and no for consistency
df["Churn"] = df["Churn"].map({1: "Yes", 0: "No"}).astype("object")
```

```
df[['tenure', 'MonthlyCharges', 'TotalCharges']]
```

Out[]:

	tenure	MonthlyCharges	TotalCharges
0	1	29.85	29.85
1	34	56.95	1889.50
2	2	53.85	108.15
3	45	42.30	1840.75
4	2	70.70	151.65
...	...	...	...
7038	24	84.80	1990.50
7039	72	103.20	7362.90
7040	11	29.60	346.45
7041	4	74.40	306.60
7042	66	105.65	6844.50

7032 rows × 3 columns

In []:

```
from sklearn.preprocessing import KBinsDiscretizer

# Discretize each column separately
disc_tenure = KBinsDiscretizer(n_bins=4, encode='ordinal', strategy='quantile')
df["tenure_bin"] = disc_tenure.fit_transform(df[["tenure"]])

disc_month = KBinsDiscretizer(n_bins=4, encode='ordinal', strategy='quantile')
df["MonthlyCharges_bin"] = disc_month.fit_transform(df[["MonthlyCharges"]])

disc_total = KBinsDiscretizer(n_bins=4, encode='ordinal', strategy='quantile')
df["TotalCharges_bin"] = disc_total.fit_transform(df[["TotalCharges"]])
```

In []:

```
df[['tenure_bin', 'MonthlyCharges_bin', 'TotalCharges_bin']]
```

Out[]:

	tenure_bin	MonthlyCharges_bin	TotalCharges_bin
0	0.0	0.0	0.0
1	2.0	1.0	2.0
2	0.0	1.0	0.0
3	2.0	1.0	2.0
4	0.0	2.0	0.0
...	...	...	...
7038	1.0	2.0	2.0
7039	3.0	3.0	3.0
7040	1.0	0.0	0.0
7041	0.0	2.0	0.0
7042	3.0	3.0	3.0

7032 rows × 3 columns

Replacing numeric attributes with their discretized versions and storing in a new dataframe

```
In [ ]: # Make a copy of original df
df2 = df.copy()

df2 = df2.drop(columns=["customerID"])

# Drop original numeric features
df2=df2.drop(['tenure', 'MonthlyCharges', 'TotalCharges'], axis=1)

df2['tenure_bin'] = df2['tenure_bin'].astype('object')
df2['MonthlyCharges_bin'] = df2['MonthlyCharges_bin'].astype('object')
df2['TotalCharges_bin'] = df2['TotalCharges_bin'].astype('object')

# df2 is now ready for modeling
df2.head()
```

Out[]: gender SeniorCitizen Partner Dependents PhoneService MultipleLines InternetService Online

0	Female	0	Yes	No	No	No	DSL
1	Male	0	No	No	Yes	No	DSL
2	Male	0	No	No	Yes	No	DSL
3	Male	0	No	No	No	No	DSL
4	Female	0	No	No	Yes	No	Fiber optic



```
In [ ]: df2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 7032 entries, 0 to 7042
Data columns (total 20 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   gender                                7032 non-null   object
1   SeniorCitizen                        7032 non-null   object
2   Partner                              7032 non-null   object
3   Dependents                           7032 non-null   object
4   PhoneService                         7032 non-null   object
5   MultipleLines                        7032 non-null   object
6   InternetService                      7032 non-null   object
7   OnlineSecurity                       7032 non-null   object
8   OnlineBackup                         7032 non-null   object
9   DeviceProtection                     7032 non-null   object
10  TechSupport                          7032 non-null   object
11  StreamingTV                          7032 non-null   object
12  StreamingMovies                      7032 non-null   object
13  Contract                             7032 non-null   object
14  PaperlessBilling                     7032 non-null   object
15  PaymentMethod                       7032 non-null   object
16  Churn                                7032 non-null   object
```

```

17 tenure_bin          7032 non-null  object
18 MonthlyCharges_bin  7032 non-null  object
19 TotalCharges_bin    7032 non-null  object
dtypes: object(20)
memory usage: 1.4+ MB

```

Count plot for target variable shows label imbalance

```

In [ ]: print(df2['Churn'].value_counts(normalize=True))
        df2['Churn'].value_counts(normalize=True).mul(100).plot(kind='bar')

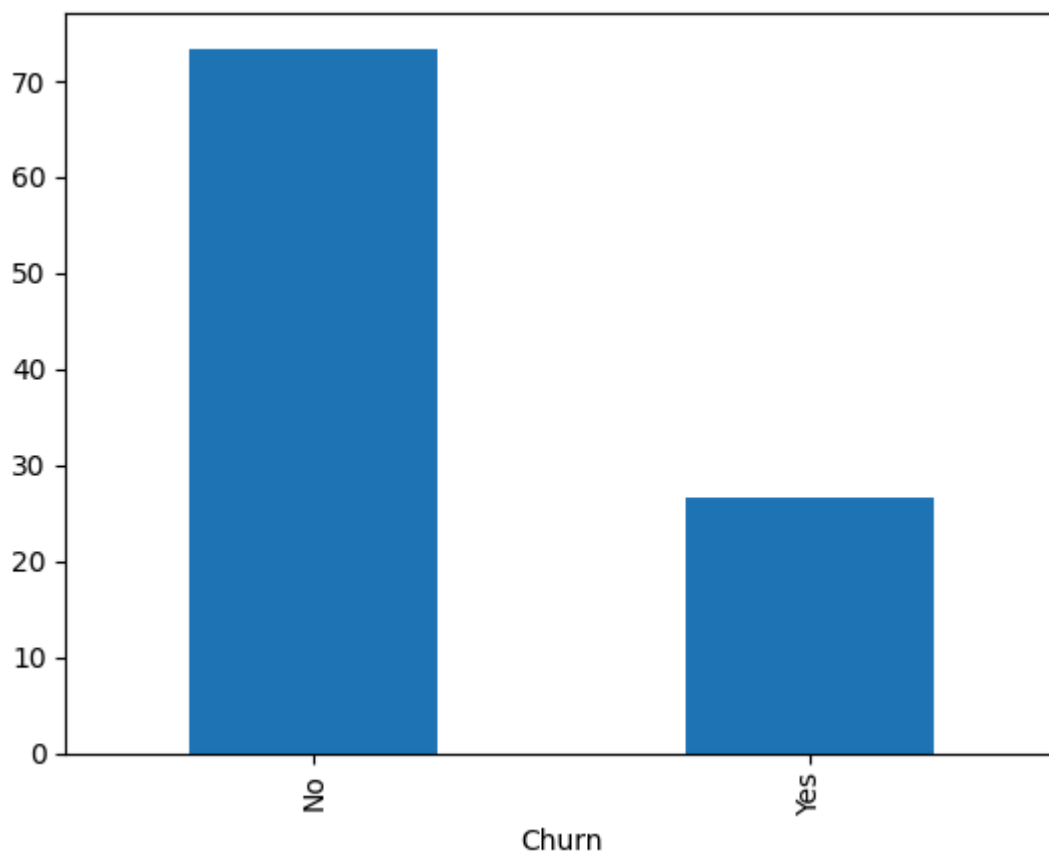
```

```

Churn
No      0.734215
Yes     0.265785
Name: proportion, dtype: float64

```

```
Out[ ]: <Axes: xlabel='Churn'>
```



Preparing Features and Target Variable

- The target column `Churn` is first converted from "Yes"/"No" to 1/0 for binary classification.
- It is then separated from the dataset to form `y` (labels), while the remaining columns become `X` (features).
- Finally, `X` is one-hot encoded using `pd.get_dummies()` to handle categorical variables.

```

In [ ]: # Convert Churn to numeric target
        df2["Churn"] = df2["Churn"].str.strip().map({"Yes": 1, "No": 0})

```

```
# Split into X and y
X = df2.drop("Churn", axis=1)
y = df2["Churn"]

# One-hot encode X only
X_encoded = pd.get_dummies(X, drop_first=True)
```

In []:

X

Out[]:

	gender	SeniorCitizen	Partner	Dependents	PhoneService	MultipleLines	InternetService	On
0	Female	0	Yes	No	No	No	DSL	
1	Male	0	No	No	Yes	No	DSL	
2	Male	0	No	No	Yes	No	DSL	
3	Male	0	No	No	No	No	DSL	
4	Female	0	No	No	Yes	No	Fiber optic	
...	...	...	...	...	...	...	...	
7038	Male	0	Yes	Yes	Yes	Yes	DSL	
7039	Female	0	Yes	Yes	Yes	Yes	Fiber optic	
7040	Female	0	Yes	Yes	No	No	DSL	
7041	Male	1	Yes	No	Yes	Yes	Fiber optic	
7042	Male	0	No	No	Yes	No	Fiber optic	

7032 rows × 19 columns



In []:

X['InternetService'].value_counts()

Out[]:

	count
InternetService	
Fiber optic	3096
DSL	2416
No	1520

dtype: int64

In []:

X_encoded

Out[]:

	gender_Male	SeniorCitizen_1	Partner_Yes	Dependents_Yes	PhoneService_Yes	MultipleLines
0	False	False	True	False	False	
1	True	False	False	False	False	True
2	True	False	False	False	False	True
3	True	False	False	False	False	False
4	False	False	False	False	False	True
...	...	...	...	...	...	...
7038	True	False	True	True	True	True
7039	False	False	True	True	True	True
7040	False	False	True	True	True	False
7041	True	True	True	False	False	True
7042	True	False	False	False	False	True

7032 rows × 29 columns



Splitting the dataset into train-test sets

In []:

```

from sklearn.model_selection import train_test_split

#Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.1)

# Final shapes:
print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)

```

Train shape: (6328, 29)

Test shape: (704, 29)

Part 1: Decision Tree

Selecting Optimal Tree Depth for Decision Tree Classifier

In []:

```

import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

train_acc = []
test_acc = []
depth_range = list(range(1, 30)) # test depths from 1 to 29

for depth in depth_range:
    clf = DecisionTreeClassifier(criterion="entropy", max_depth=depth, random_state=42)
    clf.fit(X_train, y_train)
    train_acc.append(accuracy_score(y_train, clf.predict(X_train)))
    test_acc.append(accuracy_score(y_test, clf.predict(X_test)))

```

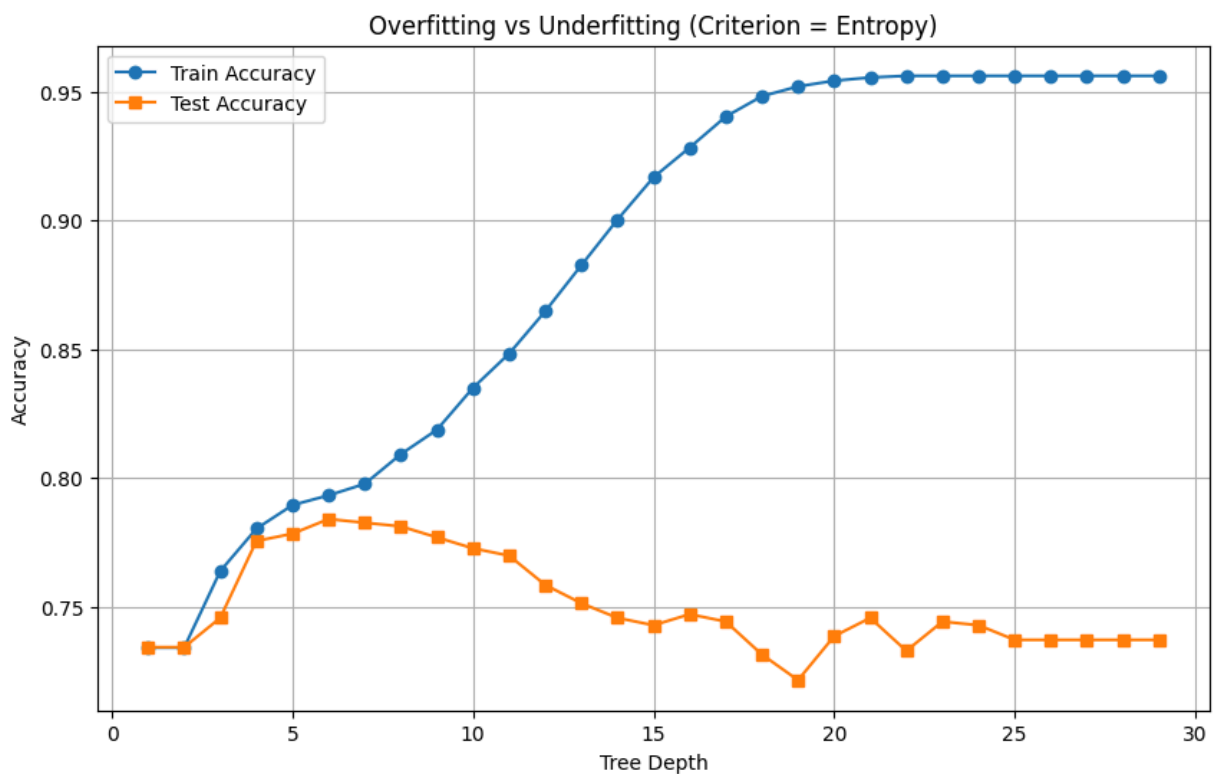
```

train_pred = clf.predict(X_train)
val_pred = clf.predict(X_test)

train_acc.append(accuracy_score(y_train, train_pred))
test_acc.append(accuracy_score(y_test, val_pred))

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(depth_range, train_acc, label="Train Accuracy", marker='o')
plt.plot(depth_range, test_acc, label="Test Accuracy", marker='s')
plt.xlabel("Tree Depth")
plt.ylabel("Accuracy")
plt.title("Overfitting vs Underfitting (Criterion = Entropy)")
plt.legend()
plt.grid(True)
plt.show()

```



```

In [ ]: df_test_acc = pd.DataFrame({"Test Accuracy": test_acc[:15]}, index=range(1, 15))
df_test_acc.T

```

```

Out[ ]:
      Test Accuracy  0.734375  0.734375  0.745739  0.775568  0.778409  0.784091  0.78267  0.78125  0.776989

```

- The best tree depth is 6, where test accuracy peaks before overfitting begins to increase.

Impact of Train-Test Ratio on Decision Tree Performance


```

In [ ]: from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
import numpy as np

# Set the best depth found from previous step
best_depth = 6

split_ratios = [0.6, 0.7, 0.8, 0.9] # train split ratios
metrics = {
    "Train Accuracy": [],
    "Test Accuracy": [],
    "Train Precision": [],
    "Test Precision": [],
    "Train Recall": [],
    "Test Recall": [],
    "Train F1": [],
    "Test F1": [],
    "Train AUC": [],
    "Test AUC": []
}

for ratio in split_ratios:
    X_train, X_test, y_train, y_test = train_test_split(
        X_encoded, y, train_size=ratio, random_state=42, stratify=y
    )

    clf = DecisionTreeClassifier(max_depth=best_depth, random_state=42, criterion='entropy')
    clf.fit(X_train, y_train)

    y_train_pred = clf.predict(X_train)
    y_test_pred = clf.predict(X_test)

    # For AUC, we need probabilities and binary classification
    y_train_prob = clf.predict_proba(X_train)[:, 1]
    y_test_prob = clf.predict_proba(X_test)[:, 1]

    metrics["Train Accuracy"].append(accuracy_score(y_train, y_train_pred))
    metrics["Test Accuracy"].append(accuracy_score(y_test, y_test_pred))

    metrics["Train Precision"].append(precision_score(y_train, y_train_pred, zero_division=0))
    metrics["Test Precision"].append(precision_score(y_test, y_test_pred, zero_division=0))

    metrics["Train Recall"].append(recall_score(y_train, y_train_pred, zero_division=0))
    metrics["Test Recall"].append(recall_score(y_test, y_test_pred, zero_division=0))

    metrics["Train F1"].append(f1_score(y_train, y_train_pred, zero_division=0))
    metrics["Test F1"].append(f1_score(y_test, y_test_pred, zero_division=0))

    metrics["Train AUC"].append(roc_auc_score(y_train, y_train_prob))
    metrics["Test AUC"].append(roc_auc_score(y_test, y_test_prob))

```

```

In [ ]: import matplotlib.pyplot as plt
import numpy as np

def plot_all_metrics_in_grid(metrics, split_ratios):
    metric_names = ["Accuracy", "Precision", "Recall", "F1", "AUC"]
    n_metrics = len(metric_names)

```

```

fig, axes = plt.subplots(2, 3, figsize=(14, 8)) # 2 rows, 3 columns
axes = axes.flatten()

for idx, metric_name in enumerate(metric_names):
    ax = axes[idx]
    ax.plot(np.array(split_ratios)*100, metrics[f"Train {metric_name}"],
            np.array(split_ratios)*100, metrics[f"Test {metric_name}"], m
    ax.set_title(f"{metric_name}")
    ax.set_xlabel("Train Size (%)")
    ax.set_ylabel(metric_name)
    ax.grid(True)
    ax.legend()

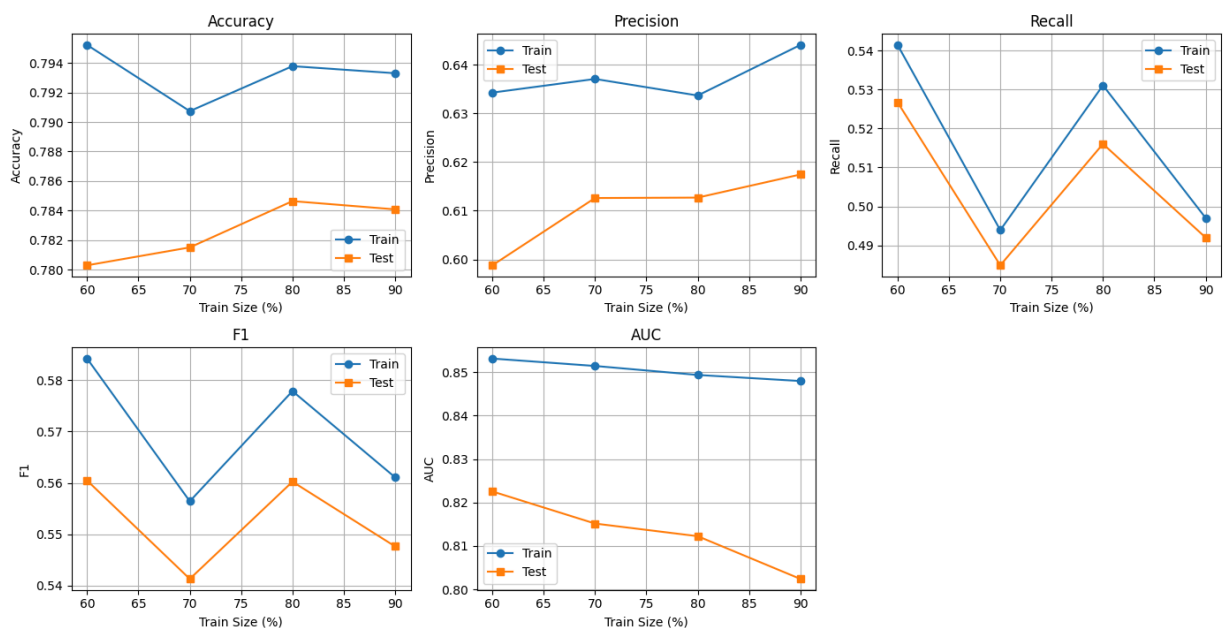
# Hide the extra subplot (6th one)
if n_metrics < len(axes):
    axes[-1].axis("off")

fig.suptitle("Model Performance vs Train/Test Split Ratio", fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.95]) # Adjust space for suptitle
plt.show()

# Call the function
plot_all_metrics_in_grid(metrics, split_ratios)

```

Model Performance vs Train/Test Split Ratio



Observations

Accuracy:

Test accuracy gradually improves as train size increases from 60% to 80 and a slight dip at 90%. The general trend is that the model benefits from more training data.

Precision:

There is a slight upward trend in test precision, highest at the 90/10 split. The model becomes more confident in its positive predictions with more training data.

Recall:

Test recall peaks around the 80/20 split and drops significantly at 90%, suggesting that higher train sizes may reduce the model's ability to detect positive cases.

F1 Score:

F1 score is highest at 80/20, showing the best trade-off between precision and recall. Both 70/30 and 90/10 are slightly worse in terms of balance.

AUC:

AUC is highest around 60/40 and then declines with the lowest at 90/10. This suggests the model's ability to distinguish between classes slightly weakens when the test set is too small.

Final Decision:

The **80/20 split** offers the most balanced performance across metrics and we choose it for final our Decision Tree classifier.

A :Final Decision Tree Training and Evaluation

In []:

```

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, classification_report
)

# 1. Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, train_size=0.8, random_state=42, stratify=y
)

# 2. Train Decision Tree (use best depth found earlier)
clf = DecisionTreeClassifier(max_depth=6, criterion="entropy", random_state=42)
clf.fit(X_train, y_train)

# 3. Predictions
y_train_pred = clf.predict(X_train)
y_test_pred = clf.predict(X_test)

# 4. Probabilities (for AUC)
y_train_prob = clf.predict_proba(X_train)[:, 1]
y_test_prob = clf.predict_proba(X_test)[:, 1]

# 5. Evaluation metrics
print("=== Train Metrics ===")
print("Accuracy:", accuracy_score(y_train, y_train_pred))
print("Precision:", precision_score(y_train, y_train_pred, zero_division=0))
print("Recall:", recall_score(y_train, y_train_pred, zero_division=0))
print("F1 Score:", f1_score(y_train, y_train_pred, zero_division=0))
print("AUC:", roc_auc_score(y_train, y_train_prob))

print("\n=== Test Metrics ===")
print("Accuracy:", accuracy_score(y_test, y_test_pred))
print("Precision:", precision_score(y_test, y_test_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_test_pred, zero_division=0))
print("F1 Score:", f1_score(y_test, y_test_pred, zero_division=0))
print("AUC:", roc_auc_score(y_test, y_test_prob))

# 6. Classification report (detailed per-class)
print("\nClassification Report:\n", classification_report(y_test, y_test_pred

```

```

=== Train Metrics ===
Accuracy: 0.7937777777777778
Precision: 0.6336791699920191
Recall: 0.5311036789297658
F1 Score: 0.5778748180494906
AUC: 0.8493592847830136

```

```

=== Test Metrics ===
Accuracy: 0.7846481876332623
Precision: 0.6126984126984127
Recall: 0.516042780748663
F1 Score: 0.5602322206095791
AUC: 0.8122388971429457

```

```

Classification Report:
              precision    recall  f1-score   support

     0       0.83         0.88         0.86         1033
     1       0.61         0.52         0.56          374

 accuracy          0.78         0.78         0.78         1407
  macro avg       0.72         0.70         0.71         1407
 weighted avg     0.78         0.78         0.78         1407

```

Observations

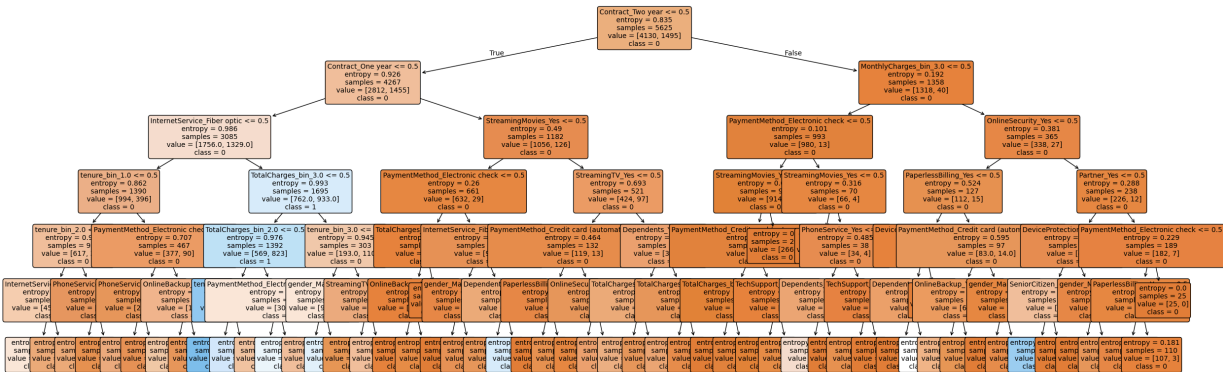
- The model achieves a test accuracy of **78.46%**, which is close to the train accuracy, indicating no major overfitting.
- Precision and recall are lower for class 1 (the positive/minority class) due to label imbalance.
- The **AUC of 0.81** on the test set shows the model has good ability to separate churn vs non-churn.
- Class 0 (non-churn) is predicted more confidently, as seen by its higher precision and recall in the classification report.
- The model generalizes well, but could benefit from improving recall on the positive class.

Visualizing the Trained Decision Tree

```

In [ ]: from sklearn.tree import plot_tree
plt.figure(figsize=(30, 10))
plot_tree(clf,
          feature_names=X_train.columns,
          class_names=[str(c) for c in clf.classes_],
          filled=True,
          rounded=True,
          fontsize=10)
plt.show()

```



```
In [ ]: # Get model predictions
y_test_pred = clf.predict(X_test)

# DataFrame combining test features, ground truth, and prediction
X_orig_train, X_orig_test = train_test_split(
    X, train_size=0.8, random_state=42, stratify=y
)

# dataframe for inspection
inspect_df = X_orig_test.copy()
inspect_df["Ground Truth"] = y_test.values
inspect_df["Prediction"] = y_test_pred

# few readable examples
inspect_df[10:20]
```

```
Out[ ]: gender SeniorCitizen Partner Dependents PhoneService MultipleLines InternetService On
```

4283	Male	1	Yes	No	Yes	No	Fiber optic
3933	Male	0	No	No	Yes	No	No
6128	Female	0	Yes	No	Yes	No	Fiber optic
6985	Male	0	Yes	Yes	No	No	DSL
4653	Female	0	Yes	Yes	No	No	DSL
346	Female	0	No	No	Yes	No	Fiber optic
6718	Male	0	No	No	Yes	Yes	Fiber optic
2039	Male	0	No	No	Yes	Yes	Fiber optic
3936	Male	0	No	No	Yes	No	DSL
4331	Male	0	No	No	Yes	Yes	No

10 rows $\times$ 21 columns

B: Decision Tree Training and Evaluation using gini as criterion

```
In [ ]: from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, classification_report
)

# 1. Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, train_size=0.8, random_state=42, stratify=y
)

# 2. Train Decision Tree (use best depth found earlier)
clf = DecisionTreeClassifier(max_depth=6, criterion="gini", random_state=42)
clf.fit(X_train, y_train)

# 3. Predictions
y_train_pred = clf.predict(X_train)
y_test_pred = clf.predict(X_test)

# 4. Probabilities (for AUC)
y_train_prob = clf.predict_proba(X_train)[:, 1]
y_test_prob = clf.predict_proba(X_test)[:, 1]

# 5. Evaluation metrics
print("=== Train Metrics ===")
print("Accuracy:", accuracy_score(y_train, y_train_pred))
print("Precision:", precision_score(y_train, y_train_pred, zero_division=0))
print("Recall:", recall_score(y_train, y_train_pred, zero_division=0))
print("F1 Score:", f1_score(y_train, y_train_pred, zero_division=0))
print("AUC:", roc_auc_score(y_train, y_train_prob))

print("\n=== Test Metrics ===")
print("Accuracy:", accuracy_score(y_test, y_test_pred))
print("Precision:", precision_score(y_test, y_test_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_test_pred, zero_division=0))
print("F1 Score:", f1_score(y_test, y_test_pred, zero_division=0))
print("AUC:", roc_auc_score(y_test, y_test_prob))

# 6. Classification report (detailed per-class)
print("\nClassification Report:\n", classification_report(y_test, y_test_pred))

=== Train Metrics ===
Accuracy: 0.8019555555555555
Precision: 0.6819484240687679
Recall: 0.47759197324414715
F1 Score: 0.5617623918174666
AUC: 0.8531291552956991

=== Test Metrics ===
Accuracy: 0.7846481876332623
Precision: 0.6263345195729537
Recall: 0.47058823529411764
F1 Score: 0.5374045801526718
AUC: 0.8128523432606343

Classification Report:
              precision    recall  f1-score   support
```

0	0.82	0.90	0.86	1033
1	0.63	0.47	0.54	374
accuracy			0.78	1407
macro avg	0.73	0.68	0.70	1407
weighted avg	0.77	0.78	0.77	1407

```

In [ ]: from sklearn.model_selection import train_test_split

#Train-test split

print('For 60/40 train-test split')

train_size=0.6
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, train_size=

train_acc = []
test_acc = []
depth_range = list(range(1, 30)) # test depths from 1 to 29

for depth in depth_range:
    clf = DecisionTreeClassifier(criterion="entropy", max_depth=depth, random
    clf.fit(X_train, y_train)

    train_pred = clf.predict(X_train)
    val_pred = clf.predict(X_test)

    train_acc.append(accuracy_score(y_train, train_pred))
    test_acc.append(accuracy_score(y_test, val_pred))

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(depth_range, train_acc, label="Train Accuracy", marker='o')
plt.plot(depth_range, test_acc, label="Test Accuracy", marker='s')
plt.xlabel("Tree Depth")
plt.ylabel("Accuracy")
plt.title("Overfitting vs Underfitting (Criterion = Entropy)")
plt.legend()
plt.grid(True)
plt.show()

df_test_acc = pd.DataFrame({"Test Accuracy": test_acc[:15]}, index=range(1, 1

depth_acc_dict = {depth: round(acc, 4) for depth, acc in zip(df_test_acc.inde
print('Value of accuracy at first 15 depth sizes:')
print(depth_acc_dict)

# Find the depth with maximum accuracy
best_depth = max(depth_acc_dict, key=depth_acc_dict.get)
print(f'\nDepth with maximum accuracy: {best_depth} (Accuracy = {depth_acc_di

# 1. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, train_size=train_size, random_state=42, stratify=y
)

# 2. Train Decision Tree (use best depth found earlier)
clf = DecisionTreeClassifier(max_depth=best_depth, criterion="entropy", rando
clf.fit(X_train, y_train)

# 3. Predictions
y_train_pred = clf.predict(X_train)
y_test_pred = clf.predict(X_test)

```



```

# 4. Probabilities (for AUC)
y_train_prob = clf.predict_proba(X_train)[: , 1]
y_test_prob = clf.predict_proba(X_test)[: , 1]

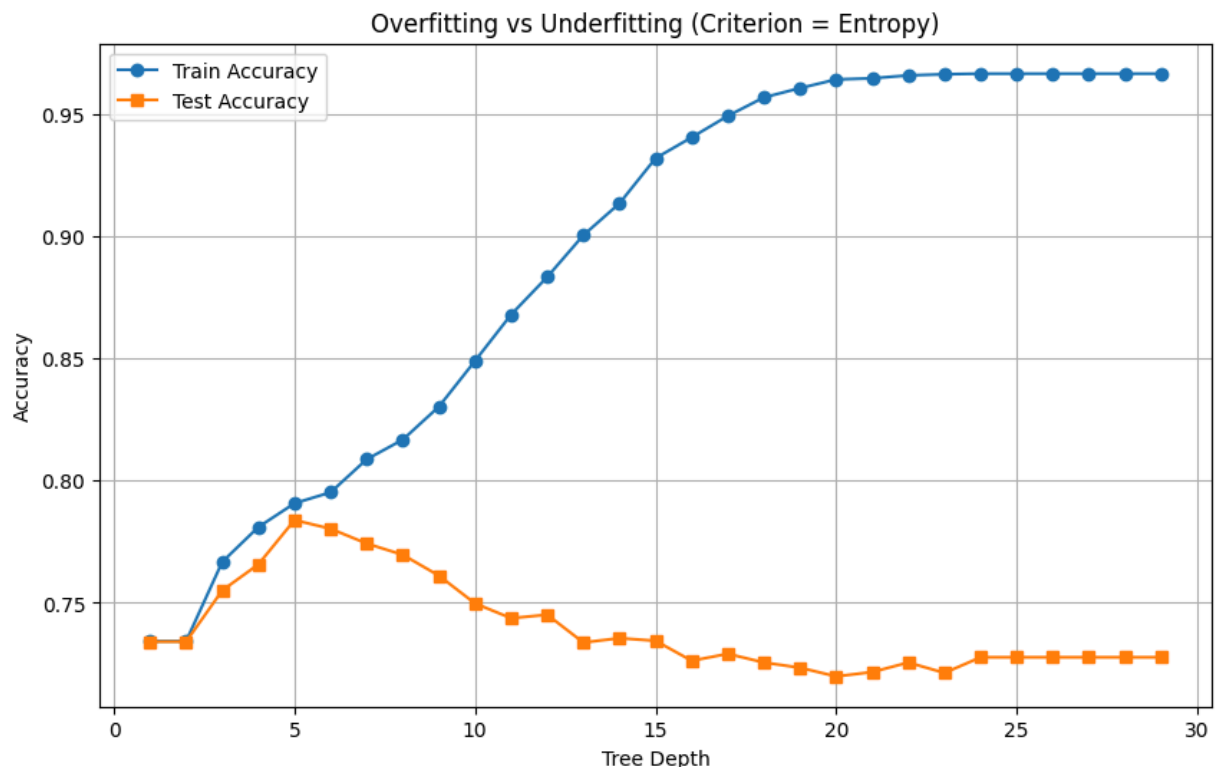
# 5. Evaluation metrics
print("=== Train Metrics ===")
print("Accuracy:", accuracy_score(y_train, y_train_pred))
print("Precision:", precision_score(y_train, y_train_pred, zero_division=0))
print("Recall:", recall_score(y_train, y_train_pred, zero_division=0))
print("F1 Score:", f1_score(y_train, y_train_pred, zero_division=0))
print("AUC:", roc_auc_score(y_train, y_train_prob))

print("\n=== Test Metrics ===")
print("Accuracy:", accuracy_score(y_test, y_test_pred))
print("Precision:", precision_score(y_test, y_test_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_test_pred, zero_division=0))
print("F1 Score:", f1_score(y_test, y_test_pred, zero_division=0))
print("AUC:", roc_auc_score(y_test, y_test_prob))

# 6. Classification report (detailed per-class)
print("\nClassification Report:\n", classification_report(y_test, y_test_pred

```

For 60/40 train-test split



Value of accuracy at first 15 depth sizes:

```
{1: 0.7341, 2: 0.7341, 3: 0.7551, 4: 0.7657, 5: 0.7839, 6: 0.7803, 7: 0.7743,
8: 0.7696, 9: 0.7611, 10: 0.7497, 11: 0.7437, 12: 0.7451, 13: 0.7337, 14: 0.7355, 15: 0.7344}
```

Depth with maximum accuracy: 5 (Accuracy = 0.7839)

=== Train Metrics ===

```
Accuracy: 0.7907086987437781
Precision: 0.6403301886792453
Recall: 0.48438893844781444
F1 Score: 0.5515490096495683
AUC: 0.8335401274685
```

=== Test Metrics ===

```
Accuracy: 0.7838606469960896
Precision: 0.6182432432432432
Recall: 0.4893048128342246
```

F1 Score: 0.5462686567164179
 AUC: 0.8150723155209696

Classification Report:

	precision	recall	f1-score	support
0	0.83	0.89	0.86	2065
1	0.62	0.49	0.55	748
accuracy			0.78	2813
macro avg	0.72	0.69	0.70	2813
weighted avg	0.77	0.78	0.78	2813

For 70/30 train-test split

```
In [ ]: from sklearn.model_selection import train_test_split

#Train-test split

print('For 70/30 train split')

train_size=0.7
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, train_size=

train_acc = []
test_acc = []
depth_range = list(range(1, 30)) # test depths from 1 to 29

for depth in depth_range:
    clf = DecisionTreeClassifier(criterion="entropy", max_depth=depth, random
    clf.fit(X_train, y_train)

    train_pred = clf.predict(X_train)
    val_pred = clf.predict(X_test)

    train_acc.append(accuracy_score(y_train, train_pred))
    test_acc.append(accuracy_score(y_test, val_pred))

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(depth_range, train_acc, label="Train Accuracy", marker='o')
plt.plot(depth_range, test_acc, label="Test Accuracy", marker='s')
plt.xlabel("Tree Depth")
plt.ylabel("Accuracy")
plt.title("Overfitting vs Underfitting (Criterion = Entropy)")
plt.legend()
plt.grid(True)
plt.show()

df_test_acc = pd.DataFrame({"Test Accuracy": test_acc[:15]}, index=range(1, 1

depth_acc_dict = {depth: round(acc, 4) for depth, acc in zip(df_test_acc.inde
print('Value of accuracy at first 15 depth sizes:')
print(depth_acc_dict)

# Find the depth with maximum accuracy
best_depth = max(depth_acc_dict, key=depth_acc_dict.get)
print(f'\nDepth with maximum accuracy: {best_depth} (Accuracy = {depth_acc_di
```

```

# 1. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, train_size=train_size, random_state=42, stratify=y
)

# 2. Train Decision Tree (use best depth found earlier)
clf = DecisionTreeClassifier(max_depth=best_depth, criterion="entropy", random_state=42)
clf.fit(X_train, y_train)

# 3. Predictions
y_train_pred = clf.predict(X_train)
y_test_pred = clf.predict(X_test)

# 4. Probabilities (for AUC)
y_train_prob = clf.predict_proba(X_train)[:, 1]
y_test_prob = clf.predict_proba(X_test)[:, 1]

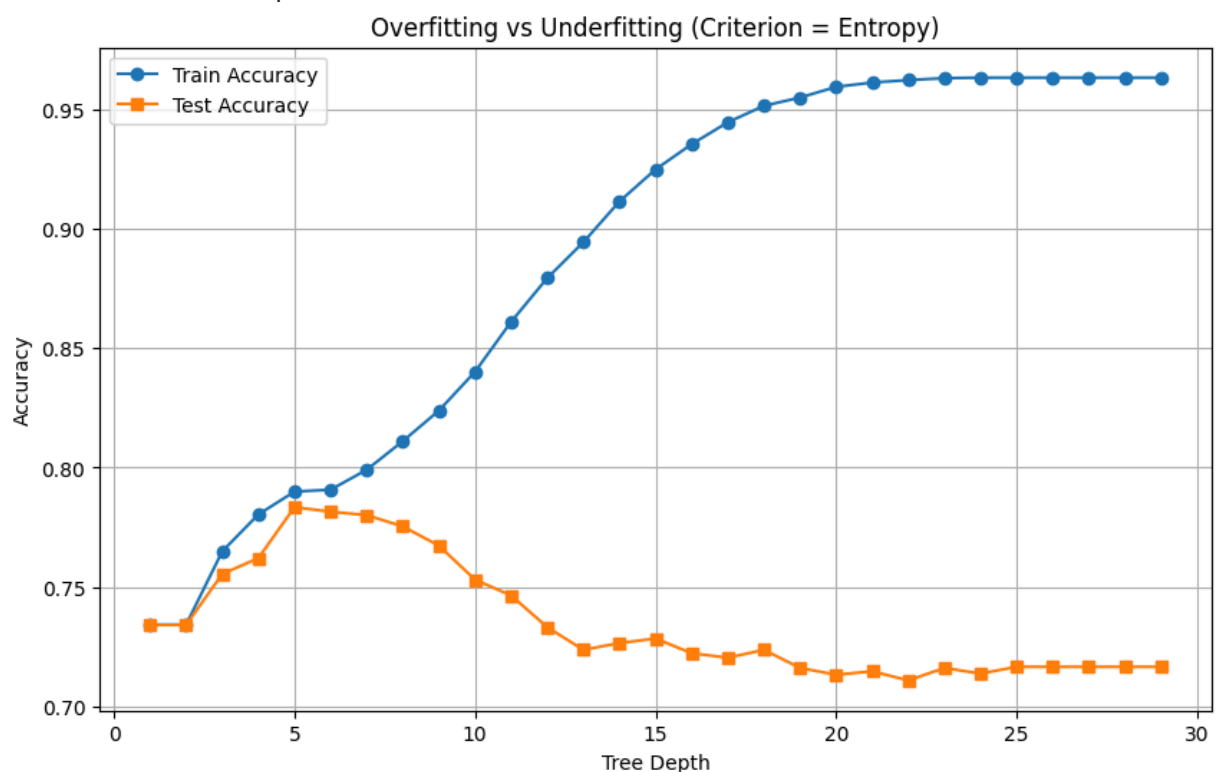
# 5. Evaluation metrics
print("=== Train Metrics ===")
print("Accuracy:", accuracy_score(y_train, y_train_pred))
print("Precision:", precision_score(y_train, y_train_pred, zero_division=0))
print("Recall:", recall_score(y_train, y_train_pred, zero_division=0))
print("F1 Score:", f1_score(y_train, y_train_pred, zero_division=0))
print("AUC:", roc_auc_score(y_train, y_train_prob))

print("\n=== Test Metrics ===")
print("Accuracy:", accuracy_score(y_test, y_test_pred))
print("Precision:", precision_score(y_test, y_test_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_test_pred, zero_division=0))
print("F1 Score:", f1_score(y_test, y_test_pred, zero_division=0))
print("AUC:", roc_auc_score(y_test, y_test_prob))

# 6. Classification report (detailed per-class)
print("\nClassification Report:\n", classification_report(y_test, y_test_pred))

```

For 70/30 train split



Value of accuracy at first 15 depth sizes:

{1: 0.7341, 2: 0.7341, 3: 0.7555, 4: 0.7621, 5: 0.7834, 6: 0.7815, 7: 0.7801, 8: 0.7754, 9: 0.7673, 10: 0.7531, 11: 0.7464, 12: 0.7332, 13: 0.7237, 14: 0.7265, 15: 0.7284}

Depth with maximum accuracy: 5 (Accuracy = 0.7834)

=== Train Metrics ===

Accuracy: 0.78992279561154

Precision: 0.637

Recall: 0.48700305810397554

F1 Score: 0.5519930675909879

AUC: 0.833631401159947

=== Test Metrics ===

Accuracy: 0.7834123222748816

Precision: 0.6181818181818182

Recall: 0.48484848484848486

F1 Score: 0.5434565434565435

AUC: 0.8129797960618604

Classification Report:

	precision	recall	f1-score	support
0	0.83	0.89	0.86	1549
1	0.62	0.48	0.54	561
accuracy			0.78	2110
macro avg	0.72	0.69	0.70	2110
weighted avg	0.77	0.78	0.77	2110

For 80/20 train-test split

```
In [ ]: from sklearn.model_selection import train_test_split

#Train-test split

print('For 80/20 train split')

train_size=0.8
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, train_size=

train_acc = []
test_acc = []
depth_range = list(range(1, 30)) # test depths from 1 to 29

for depth in depth_range:
    clf = DecisionTreeClassifier(criterion="entropy", max_depth=depth, random
    clf.fit(X_train, y_train)

    train_pred = clf.predict(X_train)
    val_pred = clf.predict(X_test)

    train_acc.append(accuracy_score(y_train, train_pred))
    test_acc.append(accuracy_score(y_test, val_pred))

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(depth_range, train_acc, label="Train Accuracy", marker='o')
plt.plot(depth_range, test_acc, label="Test Accuracy", marker='s')
plt.xlabel("Tree Depth")
```

```

plt.ylabel("Accuracy")
plt.title("Overfitting vs Underfitting (Criterion = Entropy)")
plt.legend()
plt.grid(True)
plt.show()

df_test_acc = pd.DataFrame({"Test Accuracy": test_acc[:15]}, index=range(1, 16))

depth_acc_dict = {depth: round(acc, 4) for depth, acc in zip(df_test_acc.index, df_test_acc['Test Accuracy'])}
print('Value of accuracy at first 15 depth sizes:')
print(depth_acc_dict)

# Find the depth with maximum accuracy
best_depth = max(depth_acc_dict, key=depth_acc_dict.get)
print(f'\nDepth with maximum accuracy: {best_depth} (Accuracy = {depth_acc_dict[best_depth]})')

# 1. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, train_size=train_size, random_state=42, stratify=y
)

# 2. Train Decision Tree (use best depth found earlier)
clf = DecisionTreeClassifier(max_depth=best_depth, criterion="entropy", random_state=42)
clf.fit(X_train, y_train)

# 3. Predictions
y_train_pred = clf.predict(X_train)
y_test_pred = clf.predict(X_test)

# 4. Probabilities (for AUC)
y_train_prob = clf.predict_proba(X_train)[:, 1]
y_test_prob = clf.predict_proba(X_test)[:, 1]

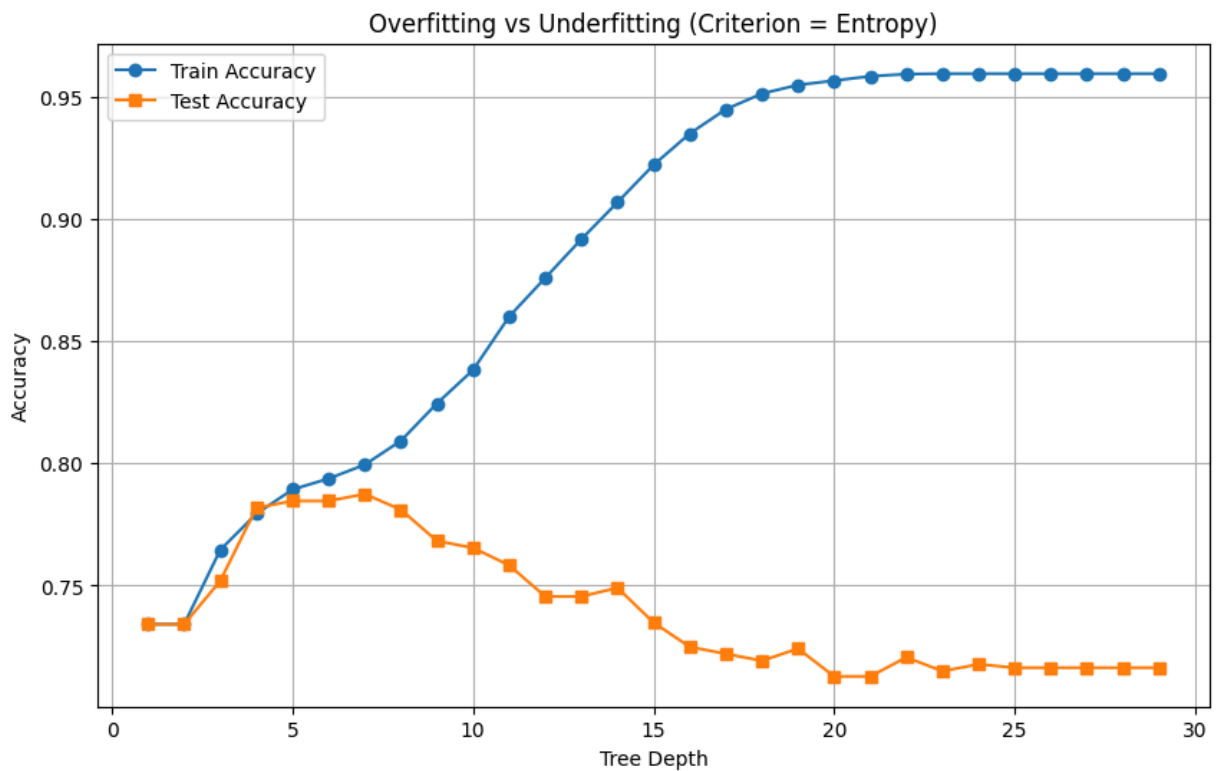
# 5. Evaluation metrics
print("=== Train Metrics ===")
print("Accuracy:", accuracy_score(y_train, y_train_pred))
print("Precision:", precision_score(y_train, y_train_pred, zero_division=0))
print("Recall:", recall_score(y_train, y_train_pred, zero_division=0))
print("F1 Score:", f1_score(y_train, y_train_pred, zero_division=0))
print("AUC:", roc_auc_score(y_train, y_train_prob))

print("\n=== Test Metrics ===")
print("Accuracy:", accuracy_score(y_test, y_test_pred))
print("Precision:", precision_score(y_test, y_test_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_test_pred, zero_division=0))
print("F1 Score:", f1_score(y_test, y_test_pred, zero_division=0))
print("AUC:", roc_auc_score(y_test, y_test_prob))

# 6. Classification report (detailed per-class)
print("\nClassification Report:\n", classification_report(y_test, y_test_pred, target_names=class_names))

```

For 80/20 train split



Value of accuracy at first 15 depth sizes:

{1: 0.7342, 2: 0.7342, 3: 0.752, 4: 0.7818, 5: 0.7846, 6: 0.7846, 7: 0.7875, 8: 0.7811, 9: 0.7683, 10: 0.7655, 11: 0.7584, 12: 0.7456, 13: 0.7456, 14: 0.7491, 15: 0.7349}

Depth with maximum accuracy: 7 (Accuracy = 0.7875)

=== Train Metrics ===

Accuracy: 0.7994666666666667
 Precision: 0.6844221105527638
 Recall: 0.45551839464882943
 F1 Score: 0.5469879518072289
 AUC: 0.8617753285770972

=== Test Metrics ===

Accuracy: 0.7874911158493249
 Precision: 0.6436781609195402
 Recall: 0.44919786096256686
 F1 Score: 0.5291338582677165
 AUC: 0.8150239942848564

Classification Report:

	precision	recall	f1-score	support
0	0.82	0.91	0.86	1033
1	0.64	0.45	0.53	374
accuracy			0.79	1407
macro avg	0.73	0.68	0.70	1407
weighted avg	0.77	0.79	0.77	1407

For 90/10 train-test split

```
In [ ]: from sklearn.model_selection import train_test_split

#Train-test split

print('For 90/10 train split')
```

```

train_size=0.6
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, train_size=

train_acc = []
test_acc = []
depth_range = list(range(1, 30)) # test depths from 1 to 29

for depth in depth_range:
    clf = DecisionTreeClassifier(criterion="entropy", max_depth=depth, random
    clf.fit(X_train, y_train)

    train_pred = clf.predict(X_train)
    val_pred = clf.predict(X_test)

    train_acc.append(accuracy_score(y_train, train_pred))
    test_acc.append(accuracy_score(y_test, val_pred))

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(depth_range, train_acc, label="Train Accuracy", marker='o')
plt.plot(depth_range, test_acc, label="Test Accuracy", marker='s')
plt.xlabel("Tree Depth")
plt.ylabel("Accuracy")
plt.title("Overfitting vs Underfitting (Criterion = Entropy)")
plt.legend()
plt.grid(True)
plt.show()

df_test_acc = pd.DataFrame({"Test Accuracy": test_acc[:15]}, index=range(1, 1

depth_acc_dict = {depth: round(acc, 4) for depth, acc in zip(df_test_acc.inde
print('Value of accuracy at first 15 depth sizes:')
print(depth_acc_dict)

# Find the depth with maximum accuracy
best_depth = max(depth_acc_dict, key=depth_acc_dict.get)
print(f'\nDepth with maximum accuracy: {best_depth} (Accuracy = {depth_acc_di

# 1. Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, train_size=train_size, random_state=42, stratify=y
)

# 2. Train Decision Tree (use best depth found earlier)
clf = DecisionTreeClassifier(max_depth=best_depth, criterion="entropy", rando
clf.fit(X_train, y_train)

# 3. Predictions
y_train_pred = clf.predict(X_train)
y_test_pred = clf.predict(X_test)

# 4. Probabilities (for AUC)
y_train_prob = clf.predict_proba(X_train)[: , 1]
y_test_prob = clf.predict_proba(X_test)[: , 1]

# 5. Evaluation metrics
print("=== Train Metrics ===")
print("Accuracy:", accuracy_score(y_train, y_train_pred))

```

```

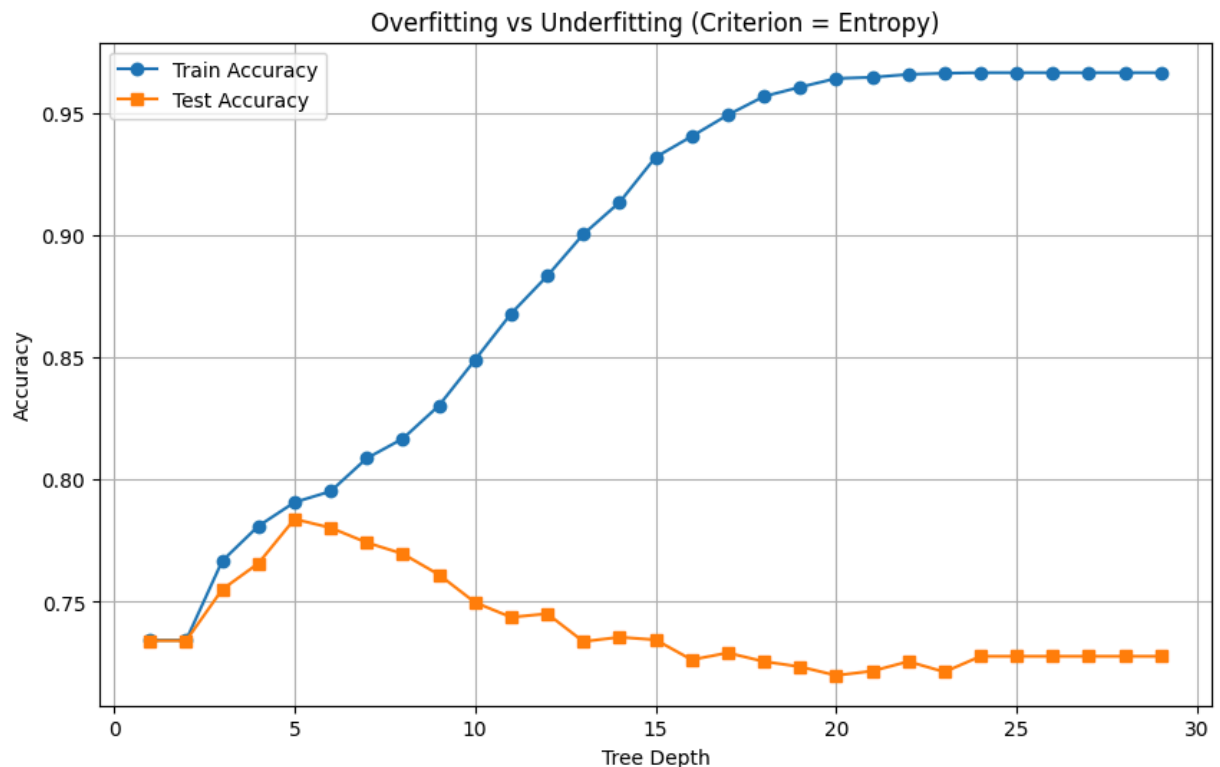
print("Precision:", precision_score(y_train, y_train_pred, zero_division=0))
print("Recall:", recall_score(y_train, y_train_pred, zero_division=0))
print("F1 Score:", f1_score(y_train, y_train_pred, zero_division=0))
print("AUC:", roc_auc_score(y_train, y_train_prob))

print("\n=== Test Metrics ===")
print("Accuracy:", accuracy_score(y_test, y_test_pred))
print("Precision:", precision_score(y_test, y_test_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_test_pred, zero_division=0))
print("F1 Score:", f1_score(y_test, y_test_pred, zero_division=0))
print("AUC:", roc_auc_score(y_test, y_test_prob))

# 6. Classification report (detailed per-class)
print("\nClassification Report:\n", classification_report(y_test, y_test_pred

```

For 90/10 train split



Value of accuracy at first 15 depth sizes:

{1: 0.7341, 2: 0.7341, 3: 0.7551, 4: 0.7657, 5: 0.7839, 6: 0.7803, 7: 0.7743, 8: 0.7696, 9: 0.7611, 10: 0.7497, 11: 0.7437, 12: 0.7451, 13: 0.7337, 14: 0.7355, 15: 0.7344}

Depth with maximum accuracy: 5 (Accuracy = 0.7839)

=== Train Metrics ===

Accuracy: 0.7907086987437781
Precision: 0.6403301886792453
Recall: 0.48438893844781444
F1 Score: 0.5515490096495683
AUC: 0.8335401274685

=== Test Metrics ===

Accuracy: 0.7838606469960896
Precision: 0.6182432432432432
Recall: 0.4893048128342246
F1 Score: 0.5462686567164179
AUC: 0.8150723155209696

Classification Report:

	precision	recall	f1-score	support
0	0.83	0.89	0.86	2065
1	0.62	0.49	0.55	748

accuracy			0.78	2813
macro avg	0.72	0.69	0.70	2813
weighted avg	0.77	0.78	0.78	2813

Part 2: Random Forest

There are two important hyperparameters in Random Forest — `n_estimators` (number of trees) and `max_features` (number of features considered at each split)

Selecting optimal `n_estimators` (number of trees) in Random Forest

- We evaluated the following range: [50, 75, 100, 150]

```
In [ ]: from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score
)
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import numpy as np

# 1. Train/test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, train_size=0.8, random_state=42, stratify=y
)

# 2. Varying n_estimators
estimator_range = [50, 75, 100, 150]
metrics_rf = {
    "Train Accuracy": [], "Test Accuracy": [],
    "Train Precision": [], "Test Precision": [],
    "Train Recall": [], "Test Recall": [],
    "Train F1": [], "Test F1": [],
    "Train AUC": [], "Test AUC": []
}

for n in estimator_range:
    clf = RandomForestClassifier(n_estimators=n, criterion='entropy', max_dep
    clf.fit(X_train, y_train)

    y_train_pred = clf.predict(X_train)
    y_test_pred = clf.predict(X_test)
    y_train_prob = clf.predict_proba(X_train)[:, 1]
    y_test_prob = clf.predict_proba(X_test)[:, 1]

    metrics_rf["Train Accuracy"].append(accuracy_score(y_train, y_train_pred))
    metrics_rf["Test Accuracy"].append(accuracy_score(y_test, y_test_pred))

    metrics_rf["Train Precision"].append(precision_score(y_train, y_train_pre
    metrics_rf["Test Precision"].append(precision_score(y_test, y_test_pred,

    metrics_rf["Train Recall"].append(recall_score(y_train, y_train_pred, zer
    metrics_rf["Test Recall"].append(recall_score(y_test, y_test_pred, zero_d

    metrics_rf["Train F1"].append(f1_score(y_train, y_train_pred, zero_divisi
```

```
metrics_rf["Test F1"].append(f1_score(y_test, y_test_pred, zero_division=0))

metrics_rf["Train AUC"].append(roc_auc_score(y_train, y_train_prob))
metrics_rf["Test AUC"].append(roc_auc_score(y_test, y_test_prob))
```

In []:

```
def plot_rf_metrics_grid(metrics, estimator_range):
    metric_names = ["Accuracy", "Precision", "Recall", "F1", "AUC"]
    fig, axes = plt.subplots(2, 3, figsize=(15, 8))
    axes = axes.flatten()

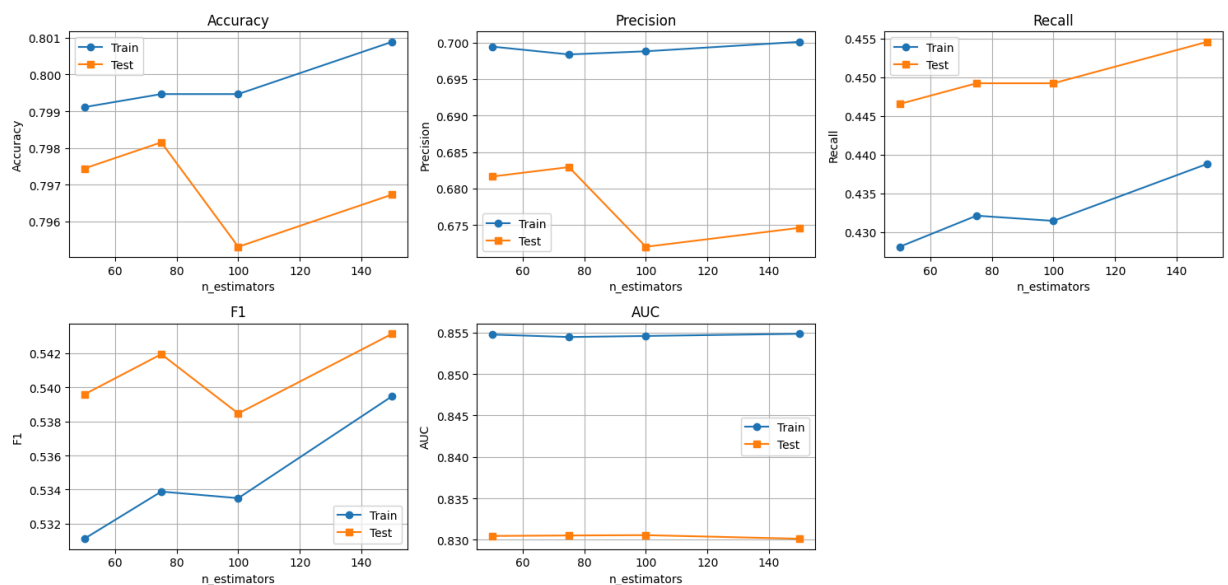
    for idx, metric in enumerate(metric_names):
        ax = axes[idx]
        ax.plot(estimator_range, metrics[f"Train {metric}"], marker='o', label='Train')
        ax.plot(estimator_range, metrics[f"Test {metric}"], marker='s', label='Test')
        ax.set_title(metric)
        ax.set_xlabel("n_estimators")
        ax.set_ylabel(metric)
        ax.grid(True)
        ax.legend()

    if len(metric_names) < len(axes):
        axes[-1].axis("off") # Hide unused subplot

    fig.suptitle("Random Forest Performance vs n_estimators", fontsize=16)
    plt.tight_layout(rect=[0, 0, 1, 0.95])
    plt.show()

# Call the plot
plot_rf_metrics_grid(metrics_rf, estimator_range)
```

Random Forest Performance vs n_estimators



Observation

- **Accuracy:** Both train and test accuracy show a dip at 100, followed by improvement in accuracy at $n_estimators = 150$.
- **Precision:** Train precision is consistent; test precision dips at 100 trees but recovers slightly by 150.
- **Recall:** Train and Test recall gradually improves as the number of estimators increases, peaking at 150.

- **F1 Score:** Test F1 is highest at `n_estimators = 150` , suggesting the best balance between precision and recall at that point.
- **AUC:** Test AUC is fairly flat, with a slight decline at 150 trees
- We use `n_estimators = 150` as it gives best overall test performance, especially in terms of recall and F1 score and comparable performance in other metrics

Selecting optimal max_features(number of features) in Random Forest

We evaluated the following range: [5, 10, 15,20,25, 29]

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score
)
from sklearn.model_selection import train_test_split

# === PARAMETERS ===
n_total_features = X.shape[1]
n_estimators = 150
max_depth = 6

# === 1. Train/test split ===
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, train_size=0.8, random_state=42, stratify=y
)

# === 2. Define max_features options ===
max_feature_range = [5, 10, 15,20,25, 29]

# === 3. Metrics Storage ===
metrics_rf = {
    "Train Accuracy": [], "Test Accuracy": [],
    "Train Precision": [], "Test Precision": [],
    "Train Recall": [], "Test Recall": [],
    "Train F1": [], "Test F1": [],
    "Train AUC": [], "Test AUC": []
}

# === 4. Loop over max_features ===
for mf in max_feature_range:
    clf = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        max_features=mf,
        criterion='entropy',
        random_state=42
    )
    clf.fit(X_train, y_train)

    y_train_pred = clf.predict(X_train)
    y_test_pred = clf.predict(X_test)
    y_train_prob = clf.predict_proba(X_train)[: , 1]
    y_test_prob = clf.predict_proba(X_test)[: , 1]
```

```

metrics_rf["Train Accuracy"].append(accuracy_score(y_train, y_train_pred))
metrics_rf["Test Accuracy"].append(accuracy_score(y_test, y_test_pred))

metrics_rf["Train Precision"].append(precision_score(y_train, y_train_pred))
metrics_rf["Test Precision"].append(precision_score(y_test, y_test_pred))

metrics_rf["Train Recall"].append(recall_score(y_train, y_train_pred, zero_division=0))
metrics_rf["Test Recall"].append(recall_score(y_test, y_test_pred, zero_division=0))

metrics_rf["Train F1"].append(f1_score(y_train, y_train_pred, zero_division=0))
metrics_rf["Test F1"].append(f1_score(y_test, y_test_pred, zero_division=0))

metrics_rf["Train AUC"].append(roc_auc_score(y_train, y_train_prob))
metrics_rf["Test AUC"].append(roc_auc_score(y_test, y_test_prob))

```

```

In [ ]: def plot_rf_metrics_grid(metrics, x_vals):
    metric_names = ["Accuracy", "Precision", "Recall", "F1", "AUC"]
    fig, axes = plt.subplots(2, 3, figsize=(15, 8))
    axes = axes.flatten()

    for idx, metric in enumerate(metric_names):
        ax = axes[idx]
        ax.plot(x_vals, metrics[f"Train {metric}"], marker='o', label="Train")
        ax.plot(x_vals, metrics[f"Test {metric}"], marker='s', label="Test")
        ax.set_title(metric)
        ax.set_xlabel("max_features")
        ax.set_ylabel(metric)
        ax.set_xticks(x_vals)
        ax.grid(True)
        ax.legend()

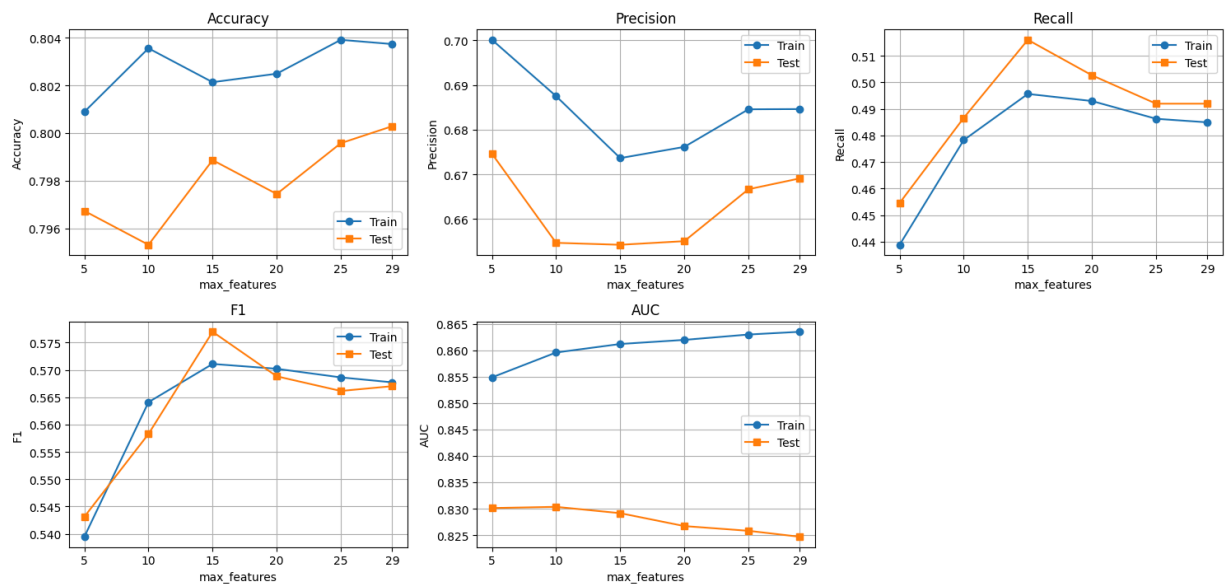
    # If extra subplot, hide it
    if len(metric_names) < len(axes):
        axes[-1].axis("off")

    fig.suptitle("Random Forest Performance vs max_features", fontsize=16)
    plt.tight_layout(rect=[0, 0, 1, 0.95])
    plt.show()

plot_rf_metrics_grid(metrics_rf, max_feature_range )

```

Random Forest Performance vs max_features



Observations

- **Accuracy:** Test accuracy is highest at **max_features = 29**, indicating the model performs best when all features are considered at each split.
- **Precision:** Test precision increases steadily with **max_features**, improving after 20 and peaking at 29.
- **Recall:** Test recall peaks at **max_features = 15**, then slightly declines — suggesting fewer false negatives around this point.
- **F1 Score:** Test F1-score is highest at **max_features = 15**, showing the best precision-recall balance.
- **AUC:** Test AUC is slightly higher at 5–10, then gradually drops as **max_features** increases, showing weaker class separation with more features.

We choose **max_features = 15**, as it gives the best trade-off between precision and recall, and achieves the highest F1-score, with a comparable performance in other metrics

C:Final Random Forest training and Evaluation

```
In [ ]:
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, classification_report
)

# === 1. Train-test split (80/20) ===
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, train_size=0.8, random_state=42, stratify=y
)

# === 2. Final Random Forest ===
clf = RandomForestClassifier(
    n_estimators=150,
```

```

max_depth=6,
max_features=15,
criterion='entropy',
random_state=42
)

clf.fit(X_train, y_train)

# === 3. Predictions ===
y_train_pred = clf.predict(X_train)
y_test_pred = clf.predict(X_test)
y_train_prob = clf.predict_proba(X_train)[:, 1]
y_test_prob = clf.predict_proba(X_test)[:, 1]

# === 4. Evaluation ===
print("=== Train Metrics ===")
print("Accuracy:", accuracy_score(y_train, y_train_pred))
print("Precision:", precision_score(y_train, y_train_pred, zero_division=0))
print("Recall:", recall_score(y_train, y_train_pred, zero_division=0))
print("F1 Score:", f1_score(y_train, y_train_pred, zero_division=0))
print("AUC:", roc_auc_score(y_train, y_train_prob))

print("\n=== Test Metrics ===")
print("Accuracy:", accuracy_score(y_test, y_test_pred))
print("Precision:", precision_score(y_test, y_test_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_test_pred, zero_division=0))
print("F1 Score:", f1_score(y_test, y_test_pred, zero_division=0))
print("AUC:", roc_auc_score(y_test, y_test_prob))

# === 5. Full classification report ===
print("\n=== Classification Report ===")
print(classification_report(y_test, y_test_pred))

```

```

=== Train Metrics ===
Accuracy: 0.8021333333333334
Precision: 0.6736363636363636
Recall: 0.4956521739130435
F1 Score: 0.5710982658959538
AUC: 0.861175508353106

```

```

=== Test Metrics ===
Accuracy: 0.798862828713575
Precision: 0.6542372881355932
Recall: 0.516042780748663
F1 Score: 0.5769805680119582
AUC: 0.8291461968929084

```

```

=== Classification Report ===

```

	precision	recall	f1-score	support
0	0.84	0.90	0.87	1033
1	0.65	0.52	0.58	374
accuracy			0.80	1407
macro avg	0.75	0.71	0.72	1407
weighted avg	0.79	0.80	0.79	1407

Observations

- The model achieves a **test accuracy of approximately 79.9%**, with minimal difference from training accuracy, indicating good generalization to unseen data.

- The **test precision (0.65)** and **F1-score (~0.58)** suggest that the model maintains a reasonable balance between correctly identifying churn cases and minimizing false positives.
- The **recall of ~0.52** indicates moderate sensitivity to detecting actual churners, which is important in the context of imbalanced datasets.
- The **AUC score of ~0.83** confirms that the model has a strong ability to distinguish between the two classes based on predicted probabilities.

D :Random Forest training and Evaluation using gini as criterion

```
In [ ]: from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, classification_report
)

# === 1. Train-test split (80/20) ===
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, train_size=0.8, random_state=42, stratify=y
)

# === 2. Final Random Forest ===
clf = RandomForestClassifier(
    n_estimators=150,
    max_depth=6,
    max_features=15,
    criterion='gini',
    random_state=42
)

clf.fit(X_train, y_train)

# === 3. Predictions ===
y_train_pred = clf.predict(X_train)
y_test_pred = clf.predict(X_test)
y_train_prob = clf.predict_proba(X_train)[:, 1]
y_test_prob = clf.predict_proba(X_test)[:, 1]

# === 4. Evaluation ===
print("=== Train Metrics ===")
print("Accuracy:", accuracy_score(y_train, y_train_pred))
print("Precision:", precision_score(y_train, y_train_pred, zero_division=0))
print("Recall:", recall_score(y_train, y_train_pred, zero_division=0))
print("F1 Score:", f1_score(y_train, y_train_pred, zero_division=0))
print("AUC:", roc_auc_score(y_train, y_train_prob))

print("\n=== Test Metrics ===")
print("Accuracy:", accuracy_score(y_test, y_test_pred))
print("Precision:", precision_score(y_test, y_test_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_test_pred, zero_division=0))
print("F1 Score:", f1_score(y_test, y_test_pred, zero_division=0))
print("AUC:", roc_auc_score(y_test, y_test_prob))

# === 5. Full classification report ===
print("\n=== Classification Report ===")
print(classification_report(y_test, y_test_pred))
```

```
=== Train Metrics ===
Accuracy: 0.8067555555555556
Precision: 0.6899441340782123
Recall: 0.4956521739130435
F1 Score: 0.5768781627092254
AUC: 0.8635675820126815

=== Test Metrics ===
Accuracy: 0.8009950248756219
Precision: 0.6666666666666666
Recall: 0.5026737967914439
F1 Score: 0.573170731707317
AUC: 0.8300340113163984

=== Classification Report ===
              precision    recall  f1-score   support

      0           0.83        0.91        0.87        1033
      1           0.67        0.50        0.57         374

 accuracy          0.80        1407
 macro avg         0.75        0.71        0.72        1407
 weighted avg      0.79        0.80        0.79        1407
```

Observations

- The model achieves a **test accuracy of approximately 80.1%**, slightly higher than the 79.9% obtained using the entropy-based version, with training accuracy closely aligned, indicating stable generalization.
- The **test precision is 0.66** and **F1-score is 0.57**, indicating a balanced ability to identify churn cases while limiting false positives.
- The **recall is 0.50**, which is marginally lower than the 0.52 observed in the entropy-based model. This suggests moderate sensitivity in detecting actual churners, which remains acceptable given the imbalanced nature of the dataset.
- The **AUC score of 0.83** confirms that the model effectively differentiates between churn and non-churn customers based on predicted probabilities.

Final Comparison of Decision Tree and Random Forest

Final Model Comparison on Test Set

Model Type	Criterion	Accuracy	Precision	Recall	F1 Score	AUC
Decision Tree (A)	Entropy	0.7846	0.6126	0.5160	0.5602	0.8122
Decision Tree (B)	Gini	0.7846	0.6263	0.4705	0.5374	0.8128
Random Forest (C)	Entropy	0.7988	0.6542	0.5160	0.5769	0.8291
Random Forest (D)	Gini	0.8009	0.6666	0.5026	0.5732	0.8300

Observations

- Both Random Forest models outperform Decision Trees across all key metrics, especially in precision, F1-score, and AUC.
- The Gini-based Random Forest achieves the **highest test accuracy (80.1%)** and **precision (0.67)**, while the entropy-based version has slightly better recall.
- Overall, Random Forest with either criterion provides more balanced and robust performance, making it a better choice for churn prediction on this dataset.